Building an AI Engine Optimization (AEO) System for Measuring How Well LLMs Answer Snowflake Developer Questions

Chanin Nantasenamat, Daniel Myers, Umesh Unnikrishnan

Developer Relations, Snowflake Inc.

Summary

AI coding assistants are now part of the Snowflake developer workflow, but there is no systematic way to measure whether those assistants give developers correct, current answers. We built an AEO benchmark that evaluates AI answer quality across 128 Snowflake developer questions spanning 32 product categories. Using a 2^4 factorial experiment design, we tested 16 combinations of four augmentation factors (domain prompt, citation instruction, agentic tools, self-critique) to isolate what actually improves answer quality. The best configuration (citation + agentic tools, no domain prompt) scored 82.3%, a 29.1 percentage-point (pp) improvement over the bare LLM baseline of 53.2%. Agentic tool access was the dominant factor (+10.9pp average), while self-critique was consistently counterproductive (−2.7pp average). These findings directly inform how Snowflake should configure its AI-powered developer tools. For product managers, the category-level analysis surfaces three documentation gap types across the 32 product categories: Debug is the weakest question type in 41% of categories (13 of 32), pointing to missing troubleshooting guides and runbooks; Compare is weakest in 28% of categories (9 of 32), indicating absent decision guidance and when-to-use content; and Implement is weakest in 25% of categories (8 of 32), reflecting incomplete how-to tutorials and code examples. These are not AI configuration problems. They are documentation coverage problems. The PM Action Framework in the Results section (Table 7) maps each gap type to a concrete documentation action and identifies the specific categories with the largest gaps.

Note on Audiences

This paper serves two distinct readers. **Engineering and platform teams** who configure AI developer tools will find the core value in the Introduction through the factorial experiment results: what levers actually improve answer quality, and by how much. **Product managers** responsible for specific Snowflake feature areas can skip directly to the Product Category Intelligence section (after the Main Effects subsection in Results): it contains per-category analysis of where AI currently struggles with developer questions about their product area, what that pattern reveals about documentation coverage gaps, and a concrete action framework tied to each gap type.

1 Introduction

In early 2026, Vercel built an AI Engine Optimization (AEO) system to track how AI coding agents reference and recommend their products. Their system measures brand

visibility: does the agent mention Vercel products? Our system asks a different question: does the agent give the *right* answer to Snowflake developer questions?

When a developer asks an AI assistant "How do I create a Cortex Search Service?", the quality of the answer depends on more than the model's training data. What actually makes an AI assistant give better answers to developer questions?

- More instructions? Bigger system prompts?
- Access to tools and documentation?
- Self-review and revision loops?
- All of the above?

We built a controlled experiment to find out.

We designed a benchmark that goes beyond brand tracking to measure multi-dimensional answer quality (correctness, completeness, recency, citation, and recommendation) against expert-authored canonical answers. Rather than testing multiple competing agents, we tested multiple augmentation configurations on a single model (`claude-opus-4-6`) to isolate the effect of each deployment lever. The result is a data-backed framework for configuring AI developer tools on the Snowflake platform.

2 Methodology

2.1 Question Bank

We authored 128 questions across 32 Snowflake product categories, with exactly 4 questions per category. Categories span the full Snowflake developer surface (Table 6) including Cortex AI Functions, Cortex Search, Cortex Agents, Cortex Code, Dynamic Tables, Snowpark, Streamlit in Snowflake, Apache Iceberg Tables, Snowflake ML, Snowpark Container Services, Native Apps Framework, Data Pipelines (Streams, Tasks, Snowpipe), Data Governance & Security, Snowflake Fundamentals & Architecture, dbt Projects on Snowflake, Semantic Views & Cortex Analyst, Database Change Management, Snowflake Postgres, and others. Each question falls into one of four test types: Explain (32), Implement (32), Debug (32), or Compare (32). For every question, we wrote a canonical answer grounded in official Snowflake documentation and defined up to five must-have factual elements (up to 640 total binary pass/fail checks). Table 1 shows one representative question per test type to illustrate the breadth of cognitive demands placed on respondents.

Table 1. One question per test type (Explain, Implement, Debug, Compare) drawn from the 128-question bank, illustrating the spectrum of cognitive demands the benchmark places on respondents. The full bank spans 32 Snowflake product categories with exactly four questions per category.

Q#	Type	Question
Q9	Explain	What are Cortex Agents and how do they orchestrate across structured and unstructured data sources?
Q26	Implement	How do I create a Snowflake-managed Iceberg table with an external volume pointing to S3?
Q79	Debug	My Snowflake Postgres instance is running slowly. How do I run health checks (cache hit ratio, bloat, vacuum status, blocking queries) and interpret the results?
Q44	Compare	When should I use streams/tasks vs. Dynamic Tables for data transformation pipelines? What factors should drive the decision?

These four examples illustrate the spectrum of reasoning the benchmark demands. An Explain question tests conceptual understanding, an Implement question requires correct syntax and documented procedure, a Debug question requires diagnostic reasoning under a realistic failure scenario, and a Compare question demands nuanced knowledge of tradeoffs between similar options. The full 128-question bank follows the same four-type structure across all 32 categories.

2.2 Scoring Rubric

Each response was scored on five dimensions using a 0–2 scale (0 = miss, 1 = partial, 2 = full). Table 2 summarises what each dimension measures.

Table 2. Five scoring dimensions used by each judge, with the evaluative question each dimension addresses. Each dimension is scored 0 (miss), 1 (partial), or 2 (full), giving a maximum of 10 points per question.

Dimension	What It Measures
Correctness	Are the facts and code accurate per current Snowflake documentation?
Completeness	Does the response cover the full answer without omitting key steps or concepts?
Recency	Does it reflect current Snowflake features and syntax rather than deprecated approaches?
Citation	Does it reference or link to official Snowflake documentation?
Recommendation	Does it suggest the Snowflake-native path when one exists?

Maximum score per question: 10 points (5 dimensions $\times$ 2). Maximum per run: 1,280 points (128 questions $\times$ 10). The final score for each response is the panel average across the three judges, expressed as a percentage of the maximum. Each question also has up to five must-have binary checks, producing a separate must-have (MH) pass rate.

2.3 Judge Panel

Every response was scored independently by three LLM judges: `openai-gpt-5.4`, `claude-opus-4-6`, and `llama4-maverick`. The final score for each response is the panel average. This design mitigates single-model scoring bias.

2.4 Scoring Pipeline: Custom vs TruLens Native

We built a custom scoring pipeline rather than using TruLens native feedback functions. This section explains the rationale and the enhancements our approach introduces.

TruLens provides a standard RAG Triad: groundedness (does the response stay faithful to retrieved context?), answer relevance (does the response address the question?), and context relevance (does the retrieved context address the question?). These three metrics are well-suited for general-purpose RAG evaluation but are not sufficient for a domain-specific developer benchmark. They do not measure factual correctness against a canonical answer, they do not check for the presence of specific must-have facts, and they do not capture dimensions like Recency (is current syntax used?) or Recommendation (does the response suggest the Snowflake-native approach?).

Our custom pipeline extends the TruLens baseline in four ways:

1. **Five-dimension rubric.** We score on Correctness, Completeness, Recency, Citation, and Recommendation using a 0–10 scale per dimension, producing a richer per-question profile than the binary RAG Triad metrics.
2. **Canonical answer grounding.** Each judge scores against an expert-authored canonical answer rather than against retrieved context alone. This catches cases where the retrieval is relevant but the response is still factually wrong or incomplete relative to the documented correct answer.
3. **Must-have binary checks.** In addition to rubric scores, each question has up to five must-have factual elements that produce a separate pass/fail signal. This makes the evaluation sensitive to the presence or absence of specific facts that a practitioner would require in a production-grade answer.
4. **Three-model judge panel.** Using three heterogeneous LLM judges (`openai-gpt-5.4`, `claude-opus-4-6`, `llama4-maverick`) and averaging their scores reduces the risk of systematic bias from any single model's preferences or blind spots.

The tradeoff is that our pipeline does not produce OpenTelemetry-compatible spans or integrate natively with Snowflake AI Observability. TruLens would provide those observability capabilities out of the box. A natural next step is to migrate the scoring pipeline to TruLens so that results are visible in Snowsight under `AI & ML > Evaluations`, while retaining our custom feedback functions as TruLens `Feedback` objects backed by the same rubric prompts.

2.5 2^4 Factorial Experiment

We tested four binary augmentation factors in all 16 possible combinations:

Table 3. Four binary augmentation factors tested in the 2^4 experiment, with the OFF and ON condition for each. All 16 combinations were evaluated, yielding 16 runs numbered in Yates order.

Factor	OFF	ON
Domain Prompt	No system message	1,800-token Snowflake product knowledge primer
Citation	Raw question only	"Cite official Snowflake docs" appended to question
Agentic Tools	Single `CORTEX.COMPLETE` call (parametric only)	Native Cortex Code session with web search, doc search, skills
Self-Critique	Single-turn generation	Two-turn generate-then-revise

Non-agentic runs (8 of 16) used `SNOWFLAKE.CORTEX.COMPLETE( 'claude-opus-4-6',...)` with a fixed 8,192-token output limit. Agentic runs (8 of 16) used native Cortex Code sessions with full tool access and no token output cap. All 16 runs used `claude-opus-4-6` exclusively as the respondent model to avoid cross-model contamination.

Domain Prompt. A 1,800-token system prompt framing the model as a Snowflake expert: "You are a Snowflake data platform expert. Provide accurate, current answers with specific SQL or Python code examples when appropriate. Reference official Snowflake documentation as the authoritative source. Recommend Snowflake-native approaches when they exist." The prompt is generic and contains no curated product knowledge, isolating whether role framing alone improves answers.

Citation Instruction. A single sentence appended directly to the user question (not a system prompt): "In your answer, reference official Snowflake documentation (docs.snowflake.com) as the authoritative source." This creates a retrieval objective with minimal intervention, making it a clean test of whether a one-line nudge outperforms a detailed expert persona.

Agentic Tools. When ON, the model runs inside Cortex Code with access to web search, bash shell, SQL execution, and file I/O. When OFF, the model is a single-turn `CORTEX.COMPLETE` call with no tools, no memory, and an 8,192-token output cap. This is the largest architectural difference in the experiment.

Self-Critique. A two-turn generate-then-revise protocol. The model first answers the question normally. A second turn then instructs: "Review your answer above for factual accuracy, completeness, and correctness against current Snowflake documentation. Revise any answers that need improvement." Questions are batched in groups of 10 for the review pass. Self-Critique was applied independently to both agentic and non-agentic conditions.

Why a factorial design? Testing factors one at a time would miss interaction effects, where two factors together behave differently than each factor alone. For example, a Domain Prompt might provide marginal benefit in isolation but actively constrain an agentic model that can retrieve its own context. A full 2^4 factorial design tests every combination and makes all such interactions directly observable from the data.

Agentic run structure. Each agentic run used native Cortex Code sessions with the task prompt: "Answer each question thoroughly and accurately using your available capabilities (skills, doc search, web search)." Where Domain Prompt was ON, the expert persona was prepended to this task. Where Citation was ON, the documentation instruction was appended to each individual question. Where Self-Critique was ON, sessions were configured to review and revise each answer before writing the final version.

Runs are numbered in Yates order: run = 1 + D + 2C + 4A + 8S, where D, C, A, S are 0 or 1. Each factor has a fixed bit weight, so any configuration is directly decodable from its run number.

Sessions were sandboxed to prevent access to canonical answers, scoring rubrics, or benchmark files.

3 Results

3.1 Overall Rankings

The 16 configurations produced scores ranging from 53.2% to 82.3%. Config abbreviations: D = Domain Prompt, C = Citation, A = Agentic, S = Self-Critique; Baseline = all factors OFF.

TL;DR: The best configuration (Citation + Agentic, no domain prompt or self-critique) scored 82.3%, a 29.1pp improvement over the bare LLM baseline of 53.2%. For a breakdown of how individual Snowflake product categories performed under each configuration, see Product Category Intelligence.

Table 4. All 16 run configurations ranked by score, from the best (C+A at 82.3%) to the Baseline (53.2%). The top six positions are held exclusively by agentic configurations, and the 29.1pp spread confirms that augmentation strategy is the dominant source of variance.

Config	Domain	Citation	Agentic	Self-Critique	Score	MH
C+A		✓	✓		**82.3%**	87.7%
D+C+A	✓	✓	✓		76.0%	82.7%
A			✓		74.4%	96.3%
D+C+A+S	✓	✓	✓	✓	73.5%	71.8%
C+A+S		✓	✓	✓	72.4%	69.0%
D+A	✓		✓		69.4%	86.0%
C		✓			67.7%	62.4%
C+S		✓		✓	67.2%	55.0%
D+C	✓	✓			66.1%	64.8%
A+S			✓	✓	66.1%	76.6%
D+C+S	✓	✓		✓	66.1%	57.8%
D+A+S	✓		✓	✓	65.4%	76.3%
D+S	✓			✓	58.4%	63.6%
D	✓				57.8%	66.1%
S				✓	56.1%	60.5%
Baseline					**53.2%**	62.7%

The engine split is clear: native Cortex Code sessions (with tool access) averaged 72.4% score and 80.8% MH, compared to 61.6% score and 61.6% MH for single `CORTEX.COMPLETE` calls. The top 6 configurations are all agentic (using Cortex Code, which has access to agentic tool calls), while the bottom 10 are predominantly non-agentic (single API calls to the LLM model without agentic tool calls). The 29.1pp score range (53.2% to 82.3%) is narrower than a preliminary 50-question pilot where the range was 37pp, reflecting that a broader question bank dampens configuration-specific variance and produces more stable rank ordering.

3.2 Model Baseline Comparison

To contextualize the factorial results, we ran two additional baseline-only runs (all four factors OFF, 128 questions, same judge panel) using `llama4-maverick` (run 17) and `openai-gpt-5.4` (run 18) as respondent models.

Table 5. Baseline scores (all factors OFF) for three respondent models, ordered by overall score. All three models also serve as judges in the panel-averaged scoring pipeline.

Model	Run	Score	Must-Have
`openai-gpt-5.4`	18	**58.0%**	69.1%
`claude-opus-4-6`	1	53.2%	62.7%
`llama4-maverick`	17	38.5%	43.6%

`openai-gpt-5.4` leads at baseline (58.0% score, 69.1% MH), edging `claude-opus-4-6` by 4.8pp on score and 6.4pp on MH. `llama4-maverick` trails substantially at 38.5% score and 43.6% MH, a 19.5pp gap below the leading model. This spread confirms that model choice independently contributes to answer quality before any configuration augmentation is applied. Notably, all three models also serve as judges in the panel-averaged scoring; their baseline scores therefore reflect respondent quality uncoupled from evaluation preferences.

The full 2^4 factorial replication across multiple respondent models remains an open item. The configuration hierarchy identified here (agentic tools dominant, self-critique counterproductive) is internally consistent for `claude-opus-4-6` but whether it holds across models is an open question.

3.3 Main Effects

The factorial design lets us compute the average impact of turning each factor ON across all 8 paired comparisons:

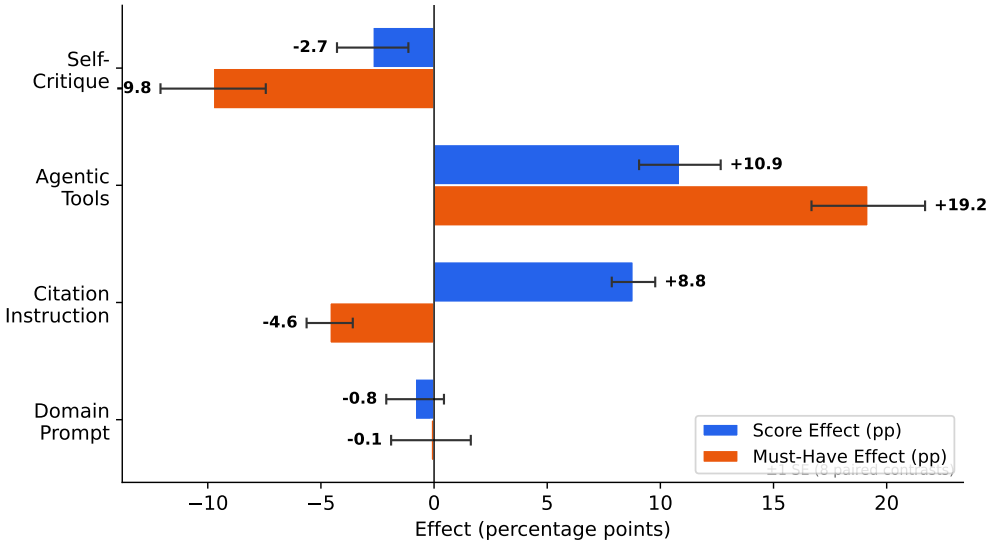

Figure 1. Plot of the average score effect and must-have effect of each augmentation factor, computed across all 8 paired comparisons in the 128-question benchmark. Agentic tools are the only factor that improves both metrics simultaneously; Self-Critique and Domain Prompt are net negative on both.

Agentic tools are the dominant factor. They improve both score and must-have compliance in every single paired comparison. The ability to search current documentation and invoke specialized skills is more valuable than any prompting strategy.

Citation instruction helps score but hurts must-have compliance. The +8.8pp score effect comes largely from the Citation dimension itself (which rises sharply when citation instruction is active, especially when combined with agentic tools that can actually retrieve and link real documentation URLs). The −4.6pp MH effect suggests that instructing the model to cite sources sometimes causes it to pad responses with references at the expense of core factual coverage.

Domain prompt is net negative. Across all 8 paired comparisons the domain prompt shows a marginal −0.8pp average score effect. A 1,800-token primer cannot meaningfully cover 32 product categories; as the question bank grows, the primer's

per-category coverage shrinks and its interference with retrieved information increases. In non-agentic conditions it provides a slight context benefit, but in agentic conditions the static system prompt actively constrains how the agent applies its retrieved content.

Self-critique is counterproductive on both metrics. It hurts score (−2.7pp) and dramatically hurts must-have pass rate (−9.8pp) across the board. The two-turn "generate then revise" pattern causes the model to second-guess correct content, introduce hedging, and sometimes remove accurate details present in the first pass.

3.4 Product Category Intelligence

Low AI scores on a product category are primarily a signal about documentation coverage, not a prompt configuration problem. Product managers cannot control how developers phrase questions to AI assistants, and they cannot control which retrieval configuration a tool uses. What they can control is the quality, completeness, and structure of the documentation that AI systems retrieve from. Table 6 reframes the category-level results around that lens, showing per-category scores broken down by question type under the best configuration (C+A).

Table 6. Per-category scores under the best configuration (C+A: citation + agentic tools), broken down by question type (Explain, Implement, Debug, Compare). The Weakest column identifies the lowest-scoring type per category, which points most directly to a specific documentation gap. All values are percentages.

Category	Overall	Expl.	Impl.	Debug	Comp.	Weakest
AI Observability & Evaluation	80.2%	84.0%	83.3%	73.3%	80.0%	***Debug***
Apache Iceberg Tables	88.3%	80.7%	97.3%	88.0%	87.3%	***Explain***
Collaboration & Data Sharing	86.8%	86.0%	88.0%	85.3%	88.0%	***Debug***
Cortex AI Function Studio	84.7%	90.0%	78.0%	84.7%	86.0%	***Implement***
Cortex AI Functions	83.0%	86.0%	99.3%	64.0%	82.7%	***Debug***
Cortex Agents	78.5%	86.0%	93.3%	52.7%	82.0%	***Debug***
Cortex Code	88.8%	86.0%	90.0%	94.0%	85.3%	***Compare***
Cortex Search	88.4%	90.7%	87.3%	92.7%	82.7%	***Compare***
Cost Management	87.5%	88.7%	79.3%	87.3%	94.7%	***Implement***
Data Clean Rooms	79.3%	84.0%	74.0%	76.7%	82.7%	***Implement***
Data Governance & Security	90.8%	86.7%	96.7%	88.0%	92.0%	***Explain***
Data Loading (COPY, Snowpipe, Streaming)	77.2%	85.3%	82.0%	54.7%	86.7%	***Debug***
Data Pipelines (Streams, Tasks, Snowpipe)	81.5%	88.7%	71.3%	81.3%	84.7%	***Implement***
Data Quality & Observability	69.2%	87.3%	52.7%	86.0%	50.7%	***Compare***
Database Change Management (DCM)	78.0%	82.0%	78.7%	67.3%	84.0%	***Debug***
Database Security	84.2%	95.3%	89.3%	82.0%	70.0%	***Compare***
Dynamic Tables	75.0%	94.7%	76.7%	64.7%	64.0%	***Compare***
Hybrid Tables	82.8%	86.0%	87.3%	78.7%	79.3%	***Debug***
Native Apps Framework	82.8%	82.0%	79.3%	83.3%	86.7%	***Implement***
Openflow	83.2%	86.7%	80.7%	83.3%	82.0%	***Implement***
SQL Performance & Optimization	82.7%	76.7%	92.7%	85.3%	76.0%	***Compare***
Semantic Views & Cortex Analyst	79.2%	80.0%	75.3%	73.3%	88.0%	***Debug***
Snowflake Fundamentals & Architecture	87.5%	88.0%	89.3%	83.3%	89.3%	***Debug***
Snowflake ML	84.3%	89.3%	76.0%	81.3%	90.7%	***Implement***
Snowflake Notebooks (Workspaces)	80.8%	84.7%	79.3%	80.7%	78.7%	***Compare***
Snowflake Postgres	61.8%	80.0%	45.3%	39.3%	82.7%	***Debug***
Snowpark	86.2%	87.3%	80.0%	93.3%	84.0%	***Implement***
Snowpark Connect & Migration	81.2%	89.3%	84.7%	78.7%	72.0%	***Compare***
Snowpark Container Services (SPCS)	85.3%	96.7%	82.0%	86.7%	76.0%	***Compare***
Snowsight	76.8%	84.0%	84.7%	53.3%	85.3%	***Debug***
Streamlit in Snowflake	86.3%	92.0%	90.7%	72.0%	90.7%	***Debug***
dbt Projects on Snowflake	90.3%	92.7%	94.0%	86.0%	88.7%	***Debug***

Debug questions are the most common weak point across categories. Debug is the weakest question type in 13 of 32 categories (41%). Compare is second (9/32) and Implement is third (8/32). Only 2 categories (Apache Iceberg Tables and Data Governance & Security) are weakest on Explain questions, suggesting conceptual documentation is relatively well-served across the platform.

The gap between Debug and the category average is often large. Cortex Agents has a 25.8pp gap (Debug 52.7% vs. 78.5% overall), Snowsight has a 23.5pp gap (Debug 53.3% vs. 76.8%), and Snowflake Postgres has a 22.5pp gap (Debug 39.3% vs. 61.8%). These are not marginal weaknesses; they indicate that even the best-configured AI assistant fails on a large share of realistic troubleshooting questions for these areas.

Diagnosing and Fixing Documentation Gaps

AI systems that answer developer questions function as documentation retrieval engines at their core. Whether through real-time retrieval (agentic runs that search current documentation) or pattern recall from training data (non-agentic API calls), the ceiling on answer quality is set by what exists in the documentation. A model cannot correctly diagnose an error whose resolution is undocumented, correctly explain a feature whose conceptual overview is absent, or correctly compare two features when no comparison guide exists. Poor scores on a question type are therefore not primarily a model failure. They are a documentation coverage signal. The model is doing the best it can with what is available to retrieve.

Each question type reflects a distinct genre of documentation. When AI fails on a question type, that failure signals that the documentation covering that genre is thin or absent for the category in question.

The four question types correspond to four primary ways developers consume documentation:

1. **Debug** questions test documentation of failure states: error messages, diagnostic steps, and recovery procedures. Poor Debug scores indicate that troubleshooting guides, runbooks, and documented error patterns are missing or sparse.
2. **Compare** questions test decision guidance documentation: when to use one feature instead of another, tradeoffs, and architectural choices. Poor Compare scores indicate that decision guides and "when to use X vs. Y" pages are absent.
3. **Implement** questions test procedural documentation: working code examples, step-by-step tutorials, and correct API syntax. Poor Implement scores indicate that how-to guides are incomplete, missing working code, or reflect outdated APIs.
4. **Explain** questions test conceptual documentation: what a feature does, how it fits the platform, and why it exists. Poor Explain scores indicate that conceptual overview pages are thin or absent.

Table 7 provides a quick reference: find the weakest question type for a category in Table 6, then read across to identify the specific documentation gap and the recommended first action.

Table 7. Diagnosing and Fixing Documentation Gaps: each weakest question type signals a distinct documentation gap and points to a specific remediation action.

Weakest	AI struggles with	Documentation gap	Recommended action
Debug	Interpreting errors, diagnostics, recovery	Troubleshooting guides and runbooks sparse or absent	Publish debug guides per feature, document common error messages and resolutions
Compare	When-to-use decisions, tradeoffs	Decision guides and "X vs Y" content absent	Publish comparison pages, add architectural decision guidance
Implement	Procedural steps, end-to-end code, syntax	How-to guides incomplete or reflect outdated APIs	Audit tutorials for completeness, add working end-to-end examples
Explain	Concepts, architecture, component relationships	Conceptual overview pages thin or absent	Strengthen concept docs, add architecture diagrams

Category-specific observations:

- **Snowflake Postgres** is the most challenging category overall (61.8%) and has severe weaknesses in both Debug (39.3%) and Implement (45.3%), suggesting that operational and procedural documentation for Postgres-specific workflows is a high-priority gap.
- **Data Quality & Observability** scores below 70% overall and is weak on both Compare (50.7%) and Implement (52.7%), indicating that architectural guidance ("when to use DMFs vs. other approaches") and step-by-step setup documentation are both thin.
- **Cortex Agents** has a 25.8pp gap between its Debug score (52.7%) and its overall score (78.5%). For a product whose core value proposition is handling complex multi-step tasks, the absence of AI-accessible troubleshooting content for agent failures is a high-risk gap.
- **Cortex AI Functions** has a Debug score of 64.0% against an overall score of 83.0% (a 19pp gap). Given the volume of developer questions about why Cortex functions return unexpected results or fail, troubleshooting guides for each function type are a direct documentation need.
- **Dynamic Tables** and **Snowsight** each have Debug scores below 55% (64.7% and 53.3% respectively). Both are high-traffic features where developers regularly encounter refresh failures, lag issues, and UI configuration errors in production.
- **Streamlit in Snowflake** scores 86.3% overall but has a 14.3pp Debug gap (72.0%), meaning AI handles Explain and Implement questions well but fails on troubleshooting. Developers hitting deployment errors or permission issues in SiS get poor AI assistance.
- **Database Security** is weak on Compare (70.0%), a 14.2pp gap from its overall score of 84.2%. Security architecture decisions ("when to use row access policies vs. column masking vs. tag-based masking") are exactly the tradeoff-heavy questions developers need guidance on.
- **Data Pipelines (Streams, Tasks, Snowpipe)** has an Implement score of 71.3% against an overall of 81.5% (a 10.2pp gap). End-to-end pipeline setup documentation covering all three components together appears to be incomplete.
- **Database Change Management (DCM)** has a Debug score of 67.3% (10.7pp gap). As a newer platform feature, troubleshooting content for DCM deployment

failures is sparse.

- **Data Loading** has a Debug score of 54.7% despite being one of the most common developer workflows on Snowflake. Diagnosing COPY INTO errors, Snowpipe ingestion failures, and streaming pipeline issues are frequent developer needs with poor AI coverage.
- **dbt Projects on Snowflake**, **Data Governance & Security**, and **Cortex Code** are the three highest-scoring categories overall, and their within-category gaps are small across all question types, indicating documentation is relatively complete for these areas.

3.5 Scoring Dimension Analysis

Citation is the dimension most sensitive to configuration. Without explicit citation instruction, models score near zero on the Citation dimension (1.1/10 in the baseline). With citation instruction plus agentic tools, it reaches 7.8/10. The other four dimensions are more stable, with agentic access providing consistent lifts across all of them. The table below shows per-dimension scores for the baseline and best run:

Table 8. Scores on each of the five evaluation dimensions for the Baseline and best configuration (C+A), revealing which aspects of answer quality are most responsive to augmentation. Citation is the most configuration-sensitive dimension, rising from 11.3% to 78.4% when citation instruction and agentic retrieval are combined.

Config	Correct.	Complete.	Recency	Citation	Recommend.
Baseline	67.0%	58.6%	67.0%	11.3%	61.8%
C+A (Best)	86.3%	79.0%	85.7%	78.4%	82.0%
Delta	+19.3pp	+20.4pp	+18.7pp	**+67.1pp**	+20.2pp

The four non-Citation dimensions (Correctness, Completeness, Recency, Recommendation) each improve by approximately 19 to 21pp under C+A, indicating that agentic retrieval provides broad quality gains across all dimensions rather than inflating any single metric. The Citation dimension's 67.1pp jump stands apart as an outlier that reflects near-total absence of citation behavior in the baseline: without explicit instruction and the ability to retrieve real documentation URLs, the model almost never cites sources.

3.6 Full Factorial Heatmap

The heatmap below shows all 16 runs against all 32 product categories, with columns grouped by whether the agentic tools factor is ON (left half) or OFF (right half). The color gradient makes the agentic divide immediately visible: the left half is predominantly green (high scores), while the right half shifts toward yellow and red.

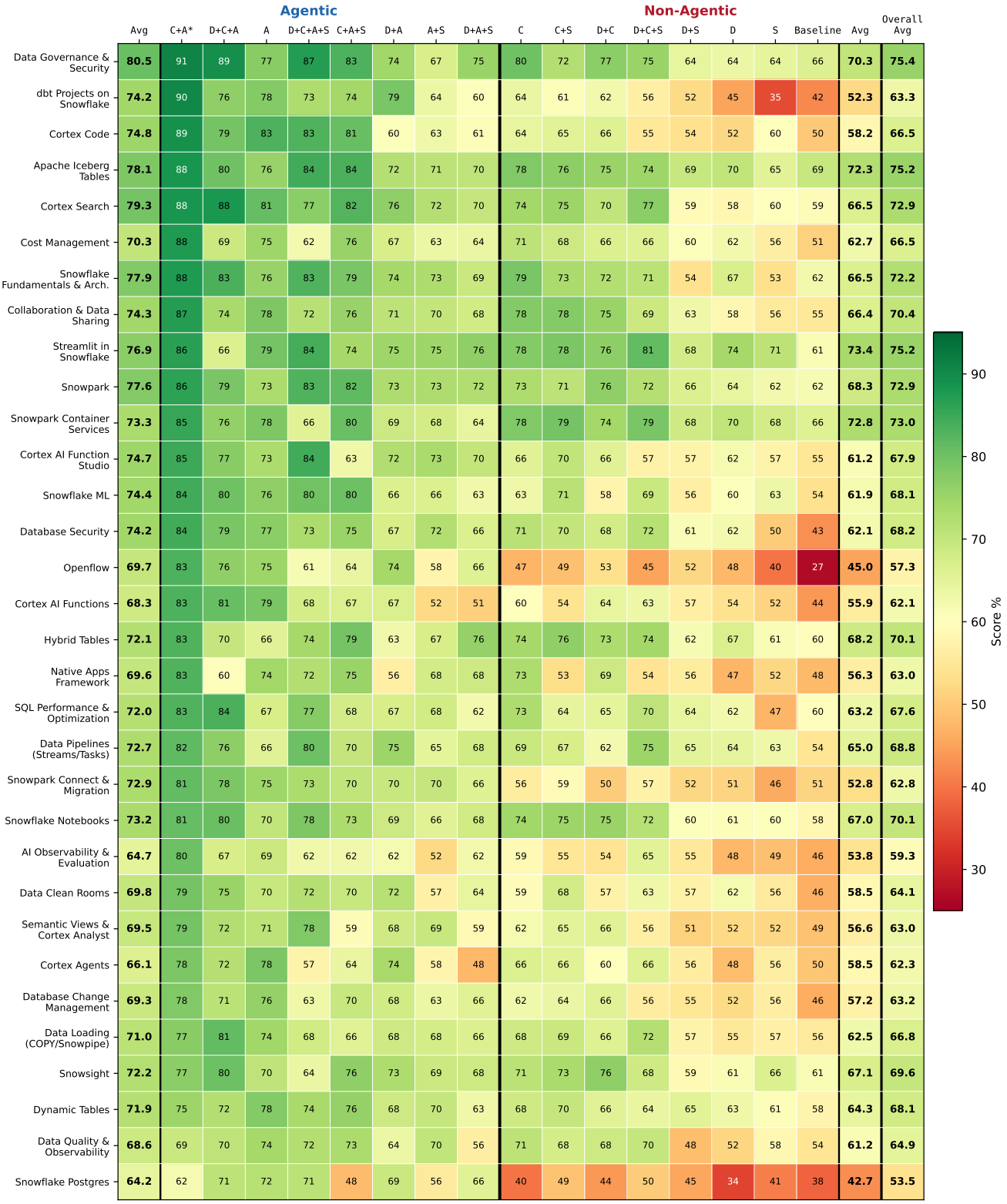

Figure 2. Category-level heatmap of all 16 factorial runs across 32 product categories. Columns are grouped by Agentic factor (left = ON, right = OFF) and sorted by average score within each group. The rightmost column shows the run average. Factor abbreviations: D = Domain Prompt, C = Citation, A = Agentic, S = Self-Critique.

Two structural patterns stand out in the heatmap. First, column color darkens sharply at the boundary between the agentic group (left half) and the non-agentic group (right half), confirming that tool access is the primary performance driver across all categories. Second, within the agentic group, configurations that include Self-Critique consistently score slightly lower than their matched counterparts without it, visible as marginally lighter cells within an otherwise high-scoring block. This pattern holds across categories, reinforcing the main-effects finding that the generate-then-revise step degrades rather than improves answer quality at this scale.

4 Conclusion

Four actionable findings emerge from this benchmark:

1. **Deploy agentic tools, not bigger prompts.** Access to current documentation and specialized skills produces larger quality improvements than any prompting strategy. The optimal configuration uses citation instruction and agentic tools with no domain prompt, achieving 82.3% versus the 53.2% baseline (a 29.1pp improvement).

Beyond the main effects, the factor interactions reveal that factors do not act independently:

2. **Pair citation instruction with agentic tools.** Citation instruction is most effective when the model can actually retrieve and link real documentation. In agentic configurations, the Citation dimension jumps from 1.1/10 to 7.8/10. In non-agentic configurations, the model can only vaguely reference documentation without providing real URLs.

3. **Remove the domain prompt from agentic configurations.** A static knowledge primer shows a marginal negative main effect (-0.8pp) and actively interferes with agentic tool use in specific combinations. The best configuration (C+A) uses no domain prompt; adding the domain prompt (D+C+A) drops the score from 82.3% to 76.0%. The domain prompt is only marginally useful in non-agentic, single-call scenarios where the model has no other source of Snowflake-specific context.

4. **Do not add self-critique steps.** The generate-then-revise pattern degrades both score (-2.7pp) and must-have compliance (-9.8pp). This is the most consistent negative finding across the full 128-question dataset: self-critique hurts in every configuration where agentic tools are present.

For product teams configuring Snowflake AI developer tools, the prescription is straightforward: give the agent tool access, instruct it to cite sources, and stay out of its way.

For product managers, the category-level findings point to a different kind of action. Three findings emerge from the per-category analysis:

5. **Debug documentation is the most widespread gap.** Debug is the weakest question type in 13 of 32 categories (41%), with gaps as large as 25.8pp (Cortex Agents), 23.5pp (Snowsight), and 22.5pp (Snowflake Postgres). These are not AI failures; they are signals that troubleshooting guides, runbooks, and documented error patterns are missing or sparse in those product areas.

6. **Compare and Implement gaps are the second and third most common failure patterns.** Compare is weakest in 9 of 32 categories (28%), indicating

absent decision guidance and "when to use X vs Y" content. Implement is weakest in 8 of 32 categories (25%), reflecting incomplete how-to tutorials and code examples. See Table 7 (PM Action Framework) in the Results section for the documentation action that maps to each gap type.

7. **Model choice matters independently of documentation.** The three-model baseline comparison shows a 19.5pp spread between the strongest respondent (`openai-gpt-5.4` at 58.0%) and the weakest (`llama4-maverick` at 38.5%) before any configuration augmentation. Teams evaluating or recommending AI coding assistants for developer use should treat model selection as a first-order decision alongside documentation investment.

Immediate first action for any PM: open Table 6 in Product Category Intelligence, find your product area, read the Weakest column, then follow the corresponding row in Table 7 (PM Action Framework) to identify the specific documentation type to invest in first.

Limitations

This benchmark has several limitations worth noting:

- **Single respondent model.** All 16 runs use `claude-opus-4-6`; results may differ for other models.
- **Question bank coverage.** The 128-question bank spans 32 categories with 4 questions each; individual category estimates carry higher variance than aggregate scores.
- **LLM-as-judge scoring.** Even with a 3-model panel, LLM judges may differ from human expert evaluation on nuanced questions. The must-have elements are binary checks that do not capture partial credit for closely related facts.
- **No TruLens integration in production scoring.** Although we built a TruLens integration (instrumented app, custom feedback functions, Snowflake connector), the 16-run factorial experiment used our custom 3-judge pipeline rather than TruLens. This means we lack standardized OpenTelemetry tracing of retrieval and generation spans, which would provide deeper observability into why agentic runs perform better. The custom pipeline also does not produce the RAG Triad metrics (groundedness, answer relevance, context relevance) that would enable direct comparison with other TruLens-evaluated systems. Migrating the scoring pipeline to TruLens would unify evaluation with Snowflake AI Observability and make results visible in Snowsight under `AI & ML > Evaluations`.

Next Steps

The immediate priorities are:

- **Automate for regression detection.** Run the benchmark on a scheduled cadence so that changes to underlying models or documentation surface as score regressions rather than surprises.
- **Replicate across models.** A three-model baseline comparison (runs 17–18, see Model Baseline Comparison) confirmed a 19.5pp spread between the strongest and weakest respondent at baseline, establishing that model choice matters independently of configuration. The full 2^4 factorial replication across all three respondent models remains open: whether the configuration hierarchy (agentic tools dominant, self-critique counterproductive) holds universally or is model-specific is the most significant remaining gap for competitive positioning use cases.

- **Expand question bank depth per category.** Four questions per category gives noisy per-category estimates (each question is 25% of the category score). Expanding to 8–12 questions per category would halve the standard error and make category-level comparisons more reliable for PM decision-making.
- **Build PM self-serve tooling.** A PM-facing Streamlit interface that shows per-category question-type scores, surfaces the documentation gap diagnosis, and links to the relevant documentation areas would close the loop between benchmark findings and documentation investment decisions.

References

- Dodds, E. and Zhou, A. (2026). "How we built AEO tracking for coding agents." *Vercel Engineering Blog*. February 9, 2026.
- Snowflake Documentation. https://docs.snowflake.com

April 13, 2026